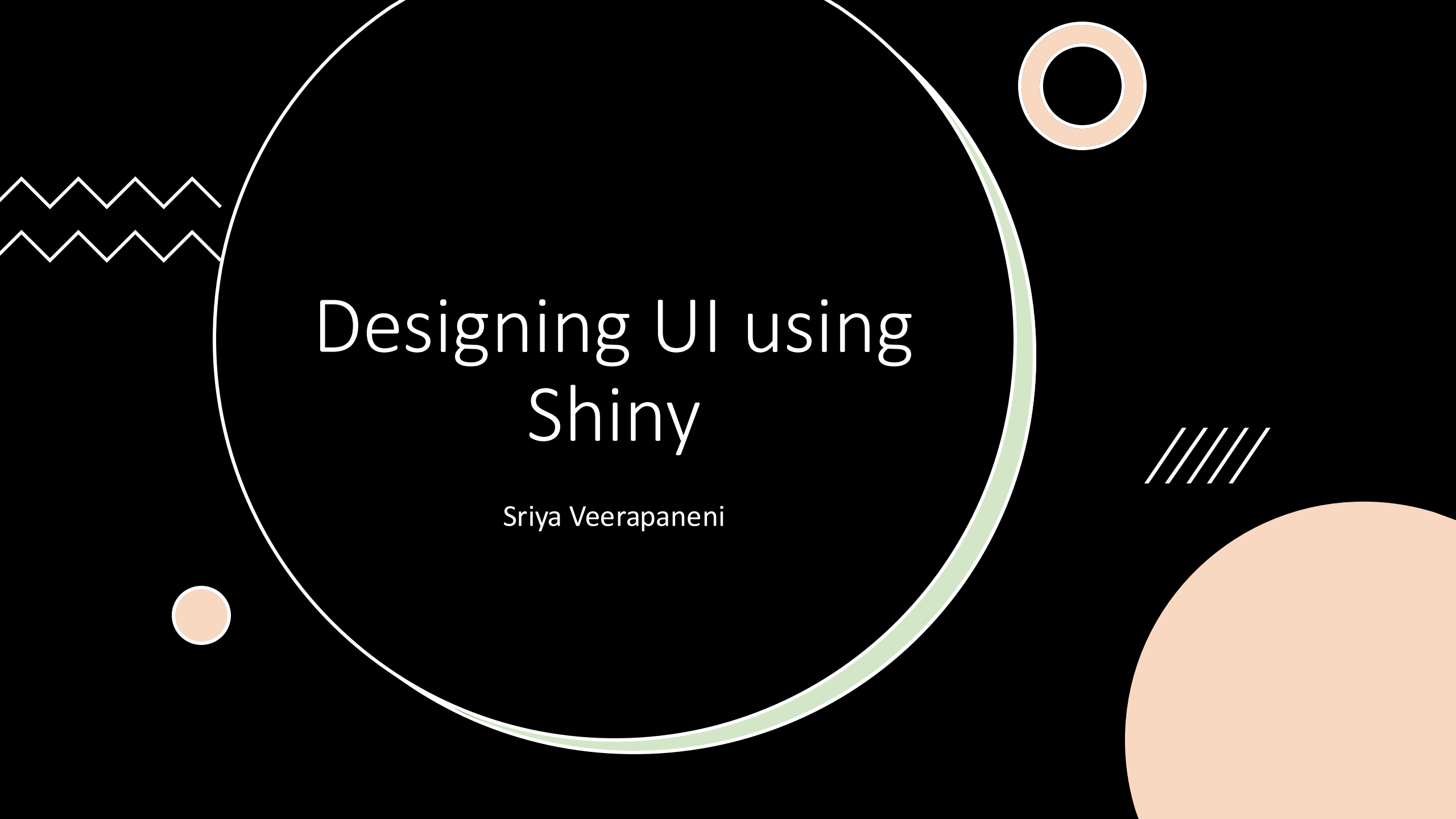




Designing UI using Shiny

Sriya Veerapaneni

What is UI



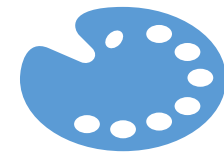
Visual designing of the
app



Defining all the elements
that go into the app



Altering layout and
graphs



Setting overall themes
and colors

FROM TO

WORK

SEARCH...

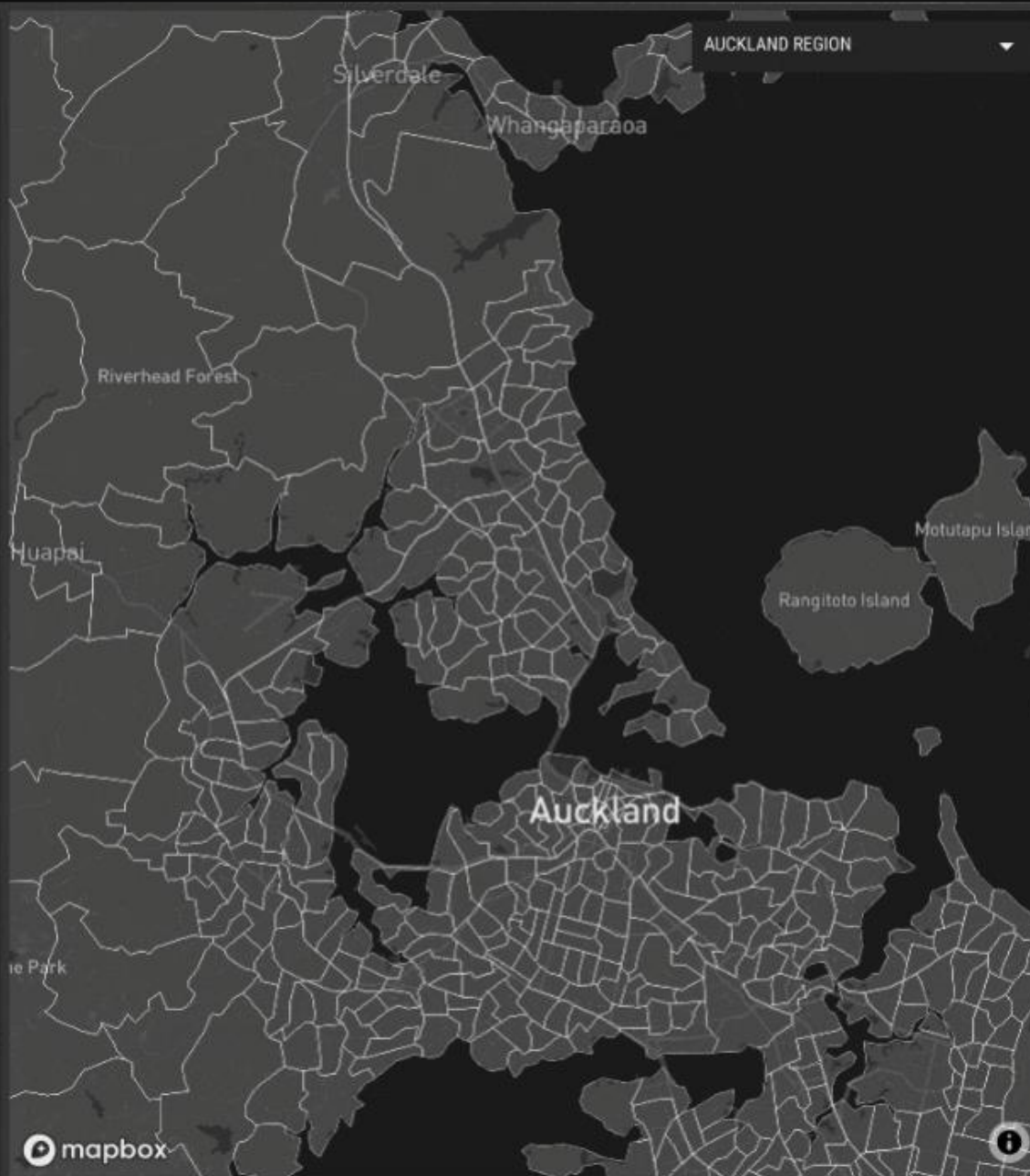
ALL DEPARTURES

AUCKLAND REGION

	🏠	🚗	🚌	🚶	🚲	Σ
🔗 Eden Terrace	198	624	414	609	30	21...
🔗 Freemans Bay	345	660	129	714	48	19...
🔗 Ormiston South	183	1563	27	9	0	19...
🔗 Morningside (Auc...	225	879	330	57	39	18...
🔗 Birkdale South	183	1182	294	30	0	18...
🔗 Parnell West	297	579	246	507	15	17...
🔗 One Tree Hill	222	1182	144	15	0	17...
🔗 Papatoetoe Cent...	135	1218	102	18	0	17...
🔗 Mount Wellinto...	156	1083	102	90	0	16...
🔗 Birkenhead West	234	999	306	18	0	16...
🔗 Point Chevalier E...	255	1098	144	24	33	16...
🔗 Greenlane North	219	951	309	75	6	16...
🔗 Avondale North (...)	150	1017	219	0	0	16...
Total	71,...	31,...	33,...	18,...	783	478,3'

1-13 of 552 suburbs

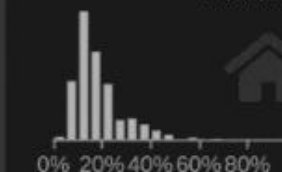
< 1 of 43 >



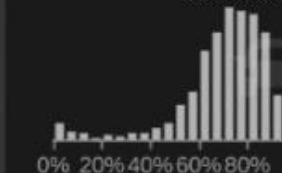
MODE OF TRAVEL

AUCKLAND REGION

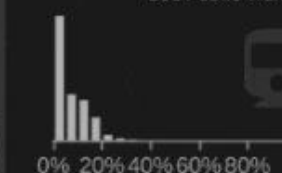
Work at Home



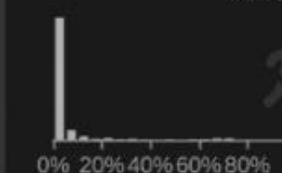
Commute by Car



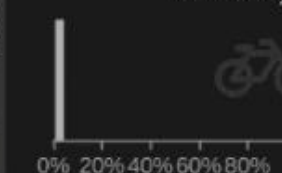
Use Public Transport



Walk or Jog



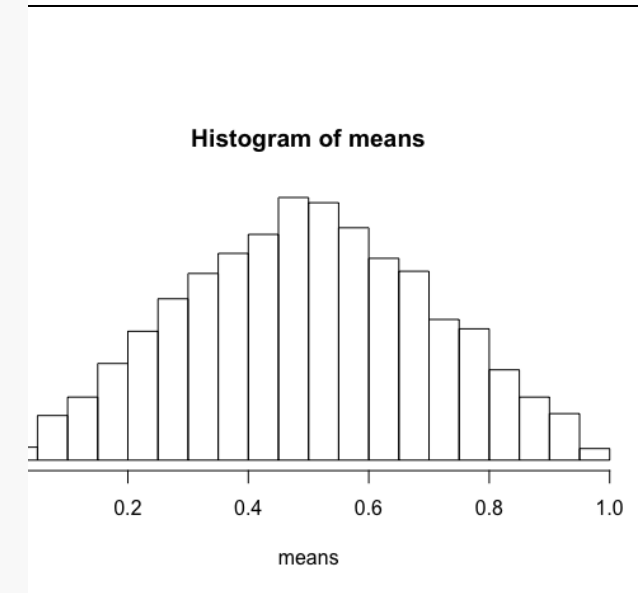
Commute by Bike



Designing L

```
ui <- fluidPage(  
  titlePanel("Central limit theorem"),  
  sidebarLayout(  
    sidebarPanel(  
      numericInput("m", "Number of sample  
    ),  
    mainPanel(  
      plotOutput("hist")  
    )  
  )  
)  
server <- function(input, output, session)  
  output$hist <- renderPlot({  
    means <- replicate(1e4, mean(runif(in  
    hist(means, breaks = 20)  
  }, res = 96)  
}
```

```
html  
  
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Central Limit Theorem</title>  
  <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>  
  <style>  
    body {  
      font-family: sans-serif;  
      margin: 40px;  
    }  
    #histogram {  
      width: 800px;  
      height: 500px;  
    }  
  </style>  
</head>  
<body>  
  
  <h2>Central Limit Theorem</h2>  
  
  <label for="samples">Number of samples (m):</label>  
  <input type="number" id="samples" value="2" min="1" max="100">  
  <button onclick="simulate()">Simulate</button>  
  
  <div id="histogram"></div>  
  
  <script>  
    function simulate() {  
      const m = parseInt(document.getElementById("samples").value);  
      const nTrials = 10000;  
      let means = [];
```



Input functions in Shiny

Common structure:

`inputId` : Identifier

`label` : Human readable label

`value` : default value set to input

```
sliderInput("min", "Limit (minimum)", value = 50, min = 0, max = 100)
```

Input functions in Shiny

- `textInput()`
 - `numericInput()`
 - `selectInput()`
 - `sliderInput()`
 - `dateInput()`
- etc....

Action button

Action

Current Value:

[1] 0
attr("class")
[1] "shinyActionButtonValue" "integer"

See Code

Single checkbox

☒ Choice A

Current Value:

[1] TRUE

See Code

Checkbox group

☒ Choice 1
☐ Choice 2
☐ Choice 3

Current Values:

[1] "1"

See Code

Date input

2014-01-01

Current Value:

[1] "2014-01-01"

See Code

Date range

2025-07-17 to 2025-07-17

Current Values:

[1] "2025-07-17" "2025-07-17"

See Code

File input

Browse... No file selected

Current Value:

NULL

See Code

Numeric input

1

Current Value:

[1] 1

See Code

Radio buttons

☒ Choice 1
☐ Choice 2
☐ Choice 3

Current Values:

[1] "1"

See Code

Select box

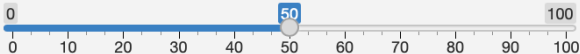
Choice 1

Current Value:

[1] "1"

See Code

Slider

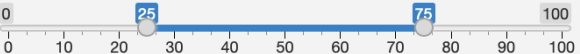


Current Value:

[1] 50

See Code

Slider range



Current Values:

[1] 25 75

See Code

Text input

Enter text...

Current Value:

[1] "Enter text..."

See Code

Outputs in Shiny

- Placeholders defined inside the **UI** that can later be called in **Server**

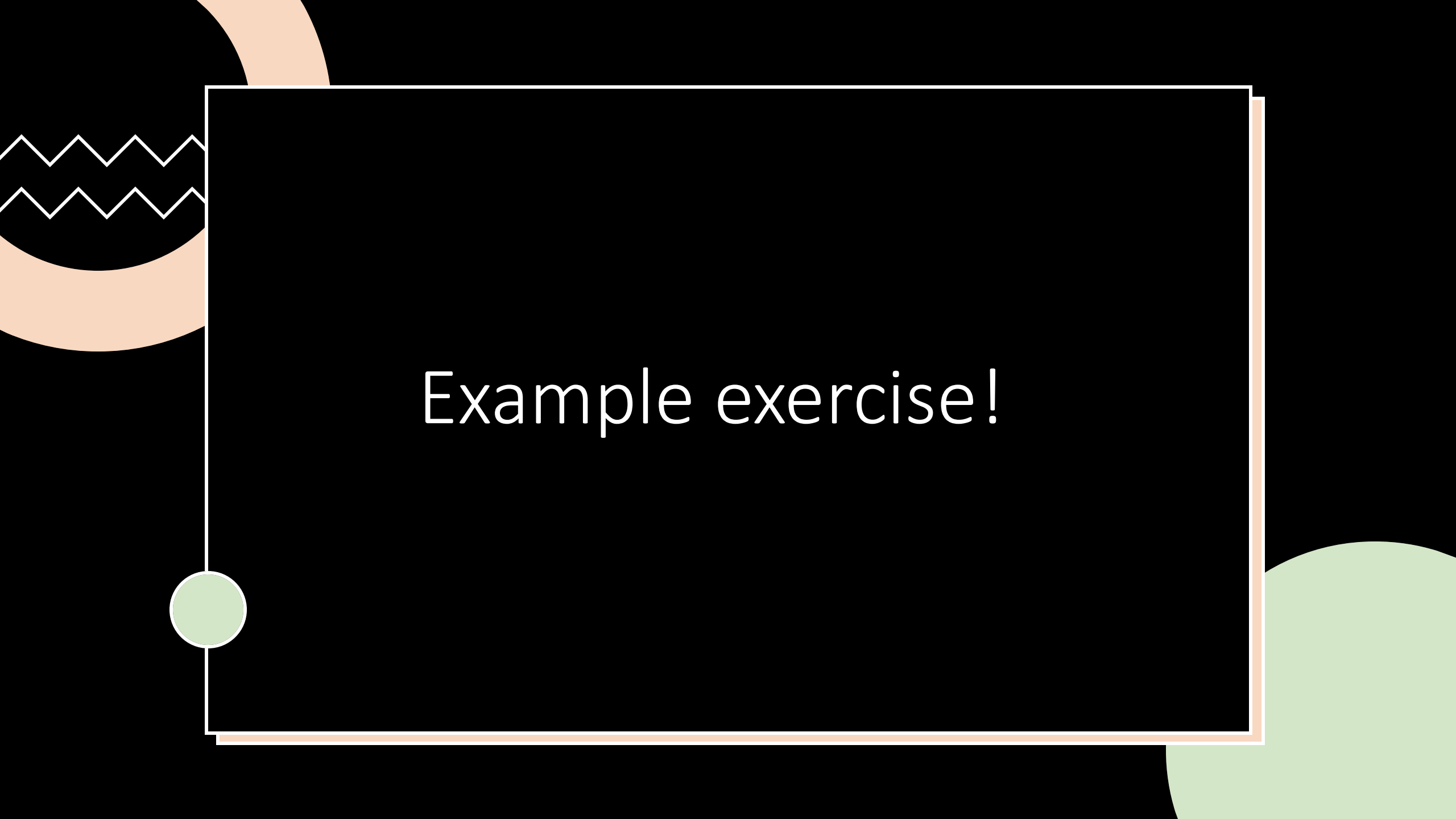
```
ui <- fluidPage(  
  textOutput("text"),  
  verbatimTextOutput("print")  
)  
server <- function(input, output, session) {  
  output$text <- renderText("hello!")  
  output$print <- renderPrint("hello!")  
}
```

hello!

```
[1] "hello!"
```


Outputs in Shiny

- `textOutput()` : `renderText()`
 - `verbatimTextOutput()` : `renderPrint()`
 - `tableOutput()`: `renderTable()`
 - `dataTableOutput()`: `renderDataTable()`
 - `plotOutput()` : `renderPlot()`
- Etc...



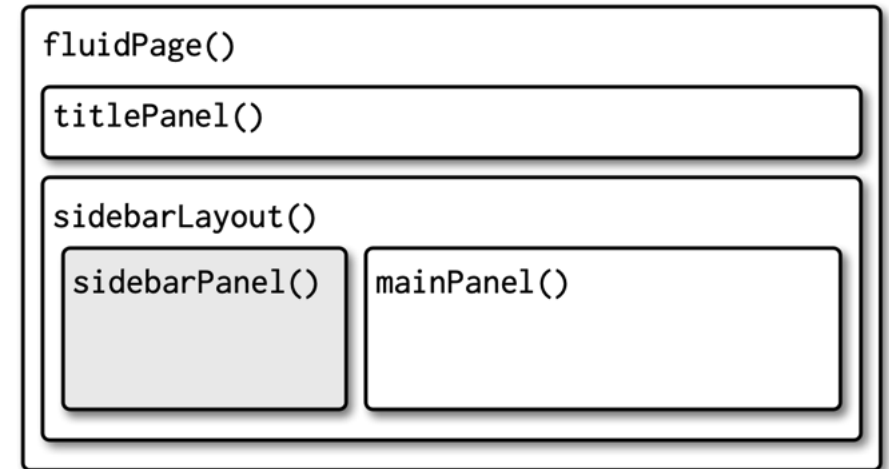
Example exercise!

Page Functions in Shiny

- `fluidPage()` – Adapts with the screen size
- `fixedPage()` – Behaves like fluid page but only till a limited fixed size
- `fillPage()` – App fills entire browser window

Layout: Sidebar Layout

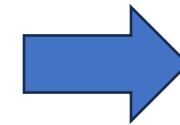
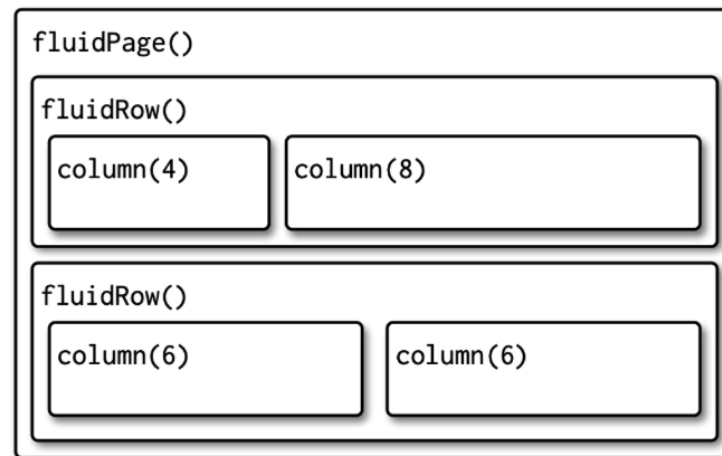
```
fluidPage(  
  titlePanel(  
    # app title/description  
  ),  
  sidebarLayout(  
    sidebarPanel(  
      # inputs  
    ),  
    mainPanel(  
      # outputs  
    )  
  )  
)
```



Custom layouts in Shiny

- Under the hood SidebarLayout() is made of a flexible Multirow layout system. This can be used to make custom layouts
- Within the fluidPage(), we can create rows with fluidRow(), and columns with column()

```
fluidPage(  
  fluidRow(  
    column(4,  
      ...  
    ),  
    column(8,  
      ...  
    )  
  ),  
  fluidRow(  
    column(6,  
      ...  
    ),  
    column(6,  
      ...  
    )  
  )  
)
```

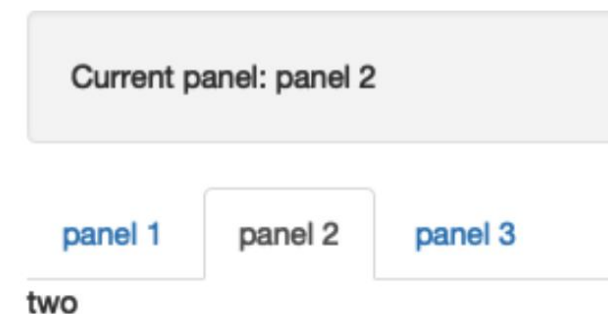
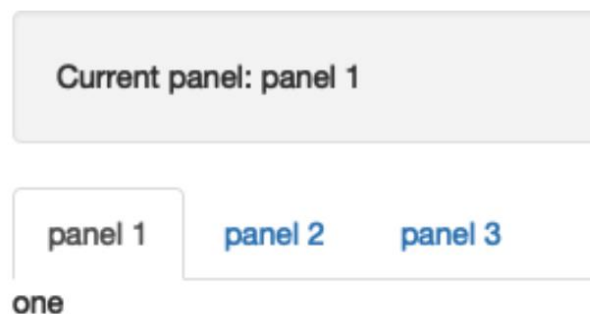


- Each row is made of 12 columns
- We can adjust column size inside the column function

Layout: Tabs

`tabsetPanel( id = "", tabPanel("panel a", "A"), ...)`

```
ui <- fluidPage(  
  sidebarLayout(  
    sidebarPanel(  
      textOutput("panel")  
    ),  
    mainPanel(  
      tabsetPanel(  
        id = "tabset",  
        tabPanel("panel 1", "one"),  
        tabPanel("panel 2", "two"),  
        tabPanel("panel 3", "three")  
      )  
    )  
  )  
)  
server <- function(input, output, session) {  
  output$panel <- renderText({  
    paste("Current panel: ", input$tabset)  
  })  
}
```



Layout: Tabs

NavbarPage(): Horizontal tabs +
navbarMenu() to set dropdowns

```
ui <- navbarPage(  
  "Page title",  
  tabPanel("panel 1", "one"),  
  tabPanel("panel 2", "two"),  
  tabPanel("panel 3", "three"),  
  navbarMenu("subpanels",  
    tabPanel("panel 4a", "four-a"),  
    tabPanel("panel 4b", "four-b"),  
    tabPanel("panel 4c", "four-c")  
  )  
)
```



navlistPanel() makes vertical tabs

```
ui <- fluidPage(  
  navlistPanel(  
    id = "tabset",  
    "Heading 1",  
    tabPanel("panel 1", "Panel one contents"),  
    "Heading 2",  
    tabPanel("panel 2", "Panel two contents"),  
    tabPanel("panel 3", "Panel three contents")  
  )  
)
```



Layout: Sidebar Layout (bslib)

```
library(shiny)
library(bslib)

# Define UI for application that draws a histogram
ui <- page_sidebar(

  title = "Hello Shiny!",

  sidebar = sidebar(
    sliderInput("bins", "Number of bins:", min = 1, value = 30, max = 50)
  ),

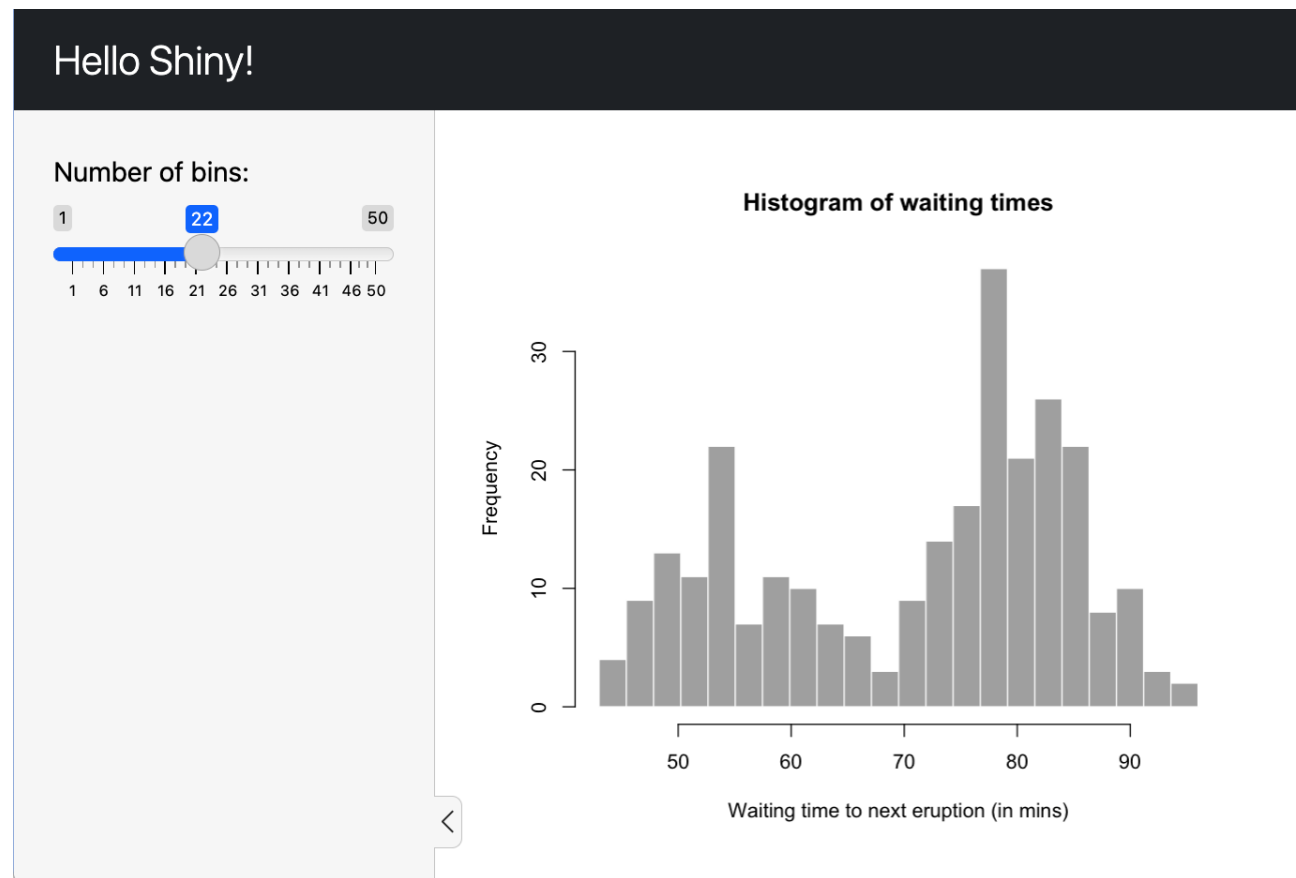
  plotOutput("distPlot")
)
```

```
# Define server logic required to draw a histogram
server <- function(input, output) {

  output$distPlot <- renderPlot({
    # generate bins based on input$bins from ui.R
    x <- faithful[, 2]
    bins <- seq(min(x), max(x), length.out = input$bins + 1)

    # draw the histogram with the specified number of bins
    hist(x, breaks = bins, col = 'darkgray', border = 'white',
        xlab = 'Waiting time to next eruption (in mins)',
        main = 'Histogram of waiting times')
  })
}

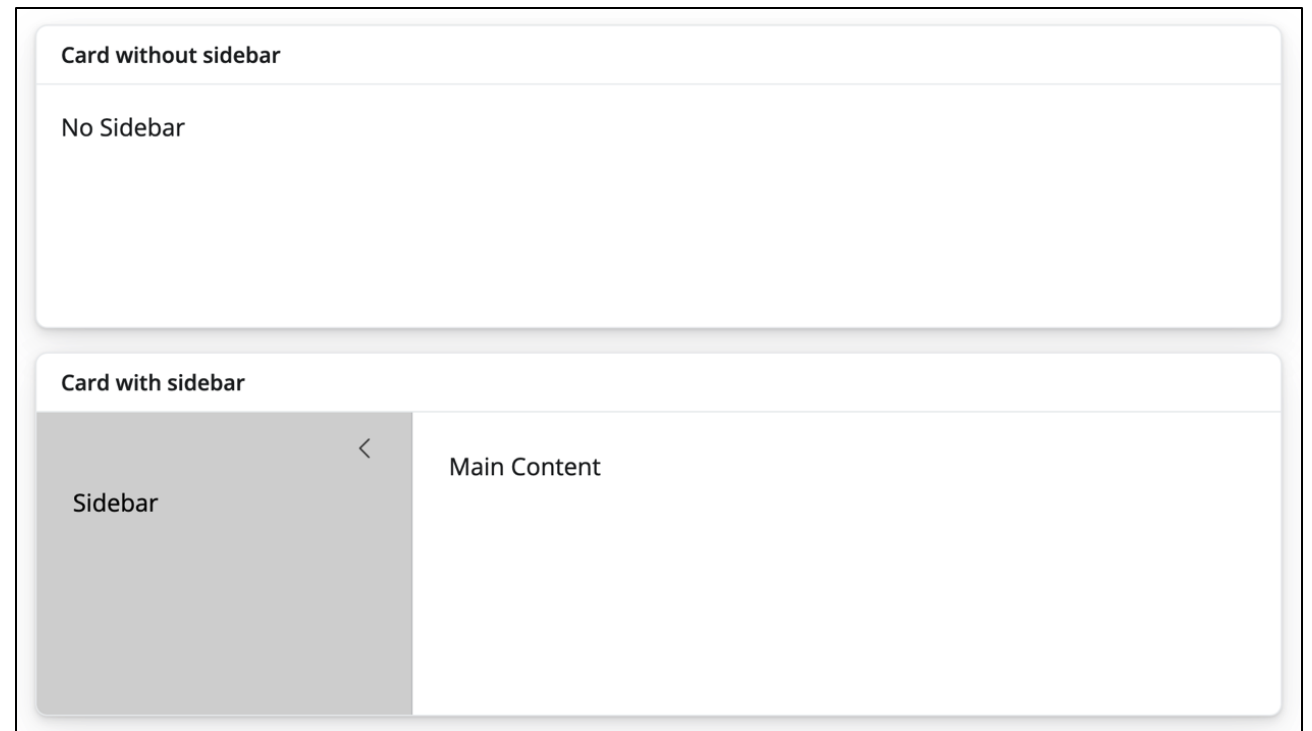
# Run the application
shinyApp(ui = ui, server = server)
```



<https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/>

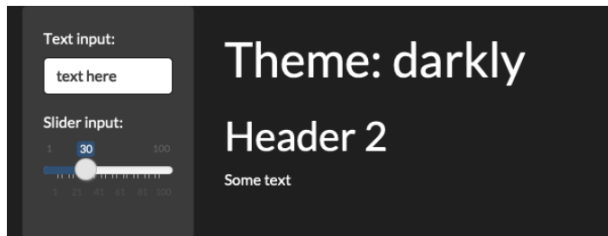
Layout: Cards (bslib)

```
ui <- page_fillable(  
  
  card(  
    card_header("Card without sidebar"),  
    "No Sidebar"  
  ),  
  
  card(  
    card_header("Card with sidebar"),  
    layout_sidebar(  
      sidebar = sidebar(  
        bg = "lightgrey",  
        "Sidebar"  
      ),  
      "Main Content"  
    )  
  )  
)
```



Themes (bslib)

```
ui <- fluidPage(  
  theme = bslib::bs_theme(bootswatch = "darkly"),
```



Theme: flatly

Header 2

Some text



Theme: sandstone

Header 2

Some text



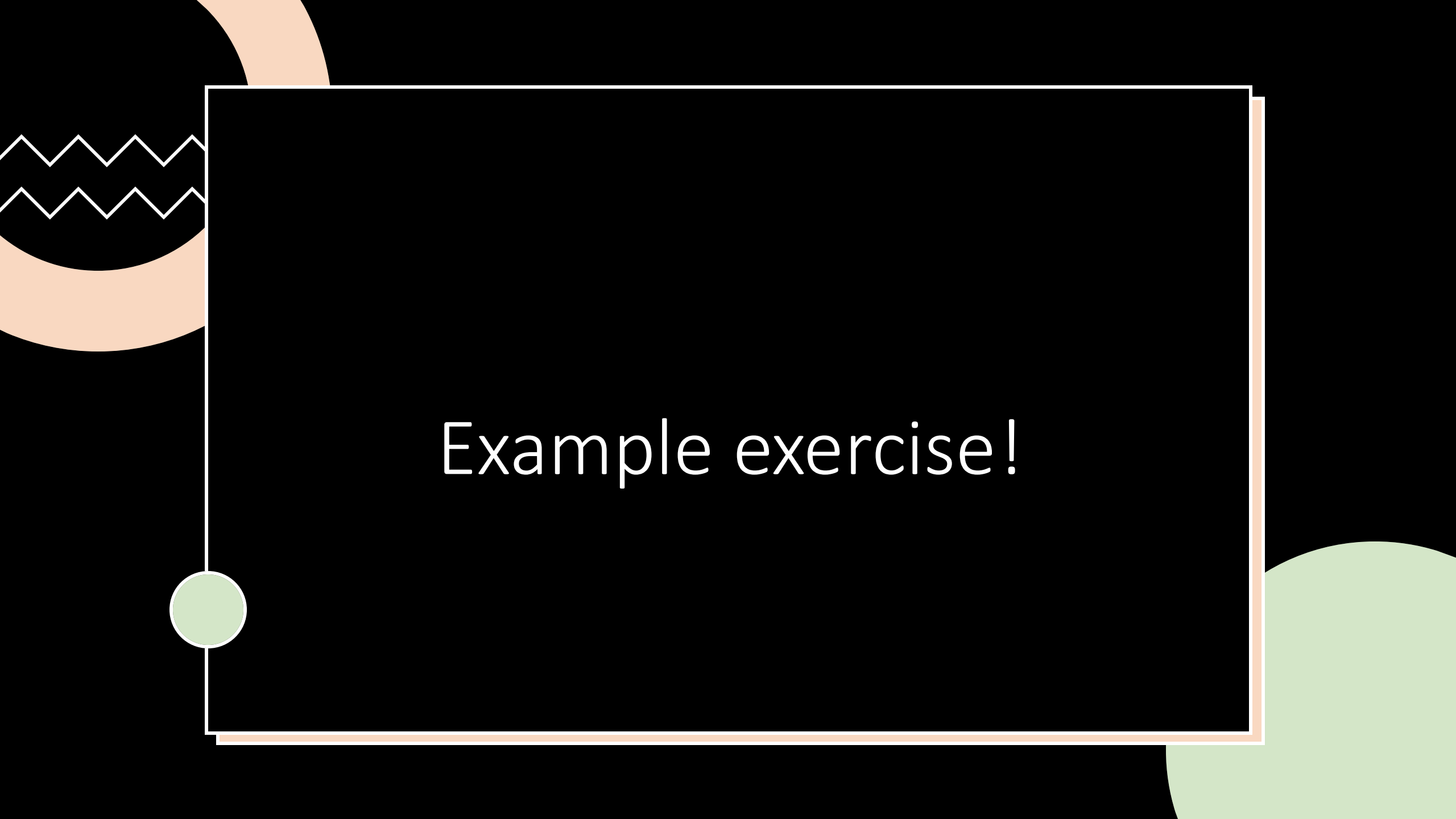
Theme: united

Header 2

Some text

Custom themes

```
theme <- bslib::bs_theme(  
  bg = "#0b3d91",  
  fg = "white",  
  base_font = "Source Sans Pro"  
)
```



Example exercise!